# Empirical Properties of Directed Acyclic Graphs in Natural Languages

A Structural Analysis of Dependency Trees vs. Random Baselines Across Six Languages

**Raunit Kumar** **230845**

Universal Dependencies Data | 6 Languages | 65,623 Sentences

**Abstract.** We test whether dependency trees in natural language are structurally different from randomly generated trees of equal size. Using real Universal Dependencies treebanks for English, Hindi, German, Japanese, Tamil, and Sanskrit (65,623 sentences total), we compare tree depth and maximum node arity against Prüfer-sequence random baselines matched sentence-by-sentence. Mann-Whitney $U$ tests with Bonferroni correction confirm: (**H1**) real trees are significantly shallower than random trees, and (**H2**) real trees show a depth-breadth tradeoff in which heads take more direct dependents than in random trees. Both hypotheses hold across all six languages (all $p < 10^{-43}$, Cohen's $|d| > 0.38$), providing strong evidence for universal structural constraints in natural language syntax.

## 1. Motivation

Natural language sentences, despite their variety, are structured in systematic ways. In **dependency grammar**, each word depends on exactly one head, forming a rooted directed acyclic graph (DAG) — a *dependency tree*. The Universal Dependencies (UD) project [9] provides harmonised dependency annotations across 100+ languages, making it possible to study structural properties at scale and cross-linguistically.

A fundamental question is whether these trees reflect genuine cognitive or communicative constraints, or whether their shape is largely arbitrary. One principled way to probe this is to compare real trees against **uniformly random trees of the same size**, trees generated with no linguistic knowledge. If real trees are structurally special, we expect systematic, statistically significant differences from this random baseline.

**Research objective:** To test whether dependency trees in natural languages exhibit structural constraints — specifically bounded depth and a depth-breadth tradeoff in node arity — that differ significantly from uniformly random trees of equal node count, and whether such constraints hold consistently across typologically diverse languages.

## 2. Hypotheses and Predictions

### 2.1 H1 — Depth Constraint

**Hypothesis:** Natural language dependency trees have significantly **lower depth** than uniformly random trees of the same number of nodes.

**Rationale:** Random trees (Prüfer baseline) produce a wide distribution of depths including very deep chain-like structures. Dependency Length Minimization (DLM; [2, 11]) predicts that speakers prefer short dependency distances, which naturally limits recursive embedding and tree height. We therefore expect real trees to be systematically shallower than random trees.

**Null hypothesis ($H_{1,0}$):** The depth distributions of real and random trees are drawn from the same population.

### 2.2 H2 — Depth-Breadth Tradeoff

**Hypothesis:** Natural language dependency trees have significantly **higher maximum node arity** than random trees of the same size, reflecting a tradeoff in which heads take more direct dependents to compensate for reduced embedding depth.

**Rationale:** A tree with $n$ nodes has exactly $n-1$ edges, so *mean* arity $= (n-1)/n$ is algebraically identical for every tree of $n$ nodes and cannot differ between real and random trees. However, *maximum* arity is free to vary. In natural language, the main verb concentrates many direct arguments (subject, object, adjuncts) at one level, keeping the tree flat. We predict that real trees have higher MaxArity as a structural consequence of H1.

**Null hypothesis ($H_{2,0}$):** The MaxArity distributions of real and random trees are drawn from the same population.

## 3. Methods

### 3.1 Data

We use six real UD treebanks in CoNLL-U format (Table 1). Sentences with fewer than 3 or more than 80 tokens are excluded. Each remaining sentence forms one directed graph: nodes = tokens, edges = dependency relations directed `head` → `dependent`.

***Table 1.*** *Languages and treebanks used. *After length filtering (3–80 tokens).*

| Language | Family | Order | Treebank | Sentences |
|---|---|---|---|---|
| English | IE / Germanic | SVO | UD_English-EWT | 12,544 |
| Hindi | IE / Indo-Aryan | SOV | UD_Hindi-HDTB | 13,306 |
| German | IE / Germanic | V2/SOV | UD_German-GSD | 13,813 |
| Japanese | Japonic | SOV | UD_Japanese-GSD | 7,050 |
| Tamil | Dravidian | SOV | UD_Tamil-TTB | 400 |
| Sanskrit | IE / Indo-Aryan | Free | UD_Sanskrit-Vedic | 21,477 |
| **Total** | | | | **65,623*** |

### 3.2 Random Baseline — Prüfer Sequence Method

For every real sentence with $n$ tokens we generate exactly one random tree also on $n$ nodes using the **Prüfer sequence method** [10]. A Prüfer sequence of length $n-2$ over $\{1, \ldots, n\}$ has a bijection to labeled trees on $n$ nodes; sampling uniformly over sequences yields uniformly random labeled trees. The undirected tree is then rooted at a random node and oriented by BFS (parent → child). This *1-to-1 pairing* controls for sentence length exactly, ensuring any observed difference is attributable to linguistic structure, not tree size.

### 3.3 Graph Metrics

**Tree Depth (H1 statistic)**

$$\text{Depth}(T) = \max_{v \in V} d(\text{root}, v)$$

Maximum directed path from root to any node, computed by single-source BFS in $O(n)$.

**Maximum Arity (H2 statistic)**

$$\text{Arity}(v) = \text{out-degree}(v) \qquad \text{MaxArity}(T) = \max_{v \in V} \text{Arity}(v)$$

We use MaxArity rather than mean arity because mean arity $= (n-1)/n$ is invariant across all trees of size $n$; MaxArity captures how concentrated dependency edges are at a single head.

### 3.4 Statistical Testing

We compare real vs. random distributions using the **Mann-Whitney $U$ test** (non-parametric; requires no normality assumption; robust to outliers). H1 uses `alternative='less'` (real depth stochastically smaller); H2 uses `alternative='greater'` (real MaxArity stochastically larger).

With $m = 12$ simultaneous tests (6 languages × 2 metrics) we apply **Bonferroni correction**:

$$\alpha' = \frac{\alpha}{m} = \frac{0.05}{12} \approx 0.00417$$

Effect size is reported as **Cohen's** $d$:

$$d = \frac{\bar{x}_{\text{real}} - \bar{x}_{\text{random}}}{s_{\text{pooled}}} \qquad s_{\text{pooled}} = \sqrt{\frac{s_1^2 + s_2^2}{2}}$$

Benchmarks: $|d| < 0.2$ negligible, 0.2–0.5 small, 0.5–0.8 medium, $> 0.8$ large.

## 4. Results

### 4.1 Descriptive Statistics

Table 2 shows the mean depth and mean MaxArity for real and random trees. In every language real trees are shallower and have higher MaxArity than random counterparts, consistent with both hypotheses. Sanskrit shows the smallest differences, reflecting its free word order and rich morphological case system that relaxes syntactic proximity constraints.

***Table 2.*** *Mean depth and mean MaxArity for real vs. random trees.* $\Delta$ *= Real − Random.*

| Language | $\text{Depth}_{\text{real}}$ | $\text{Depth}_{\text{rand}}$ | ΔDepth | $\text{MaxAr}_{\text{real}}$ | $\text{MaxAr}_{\text{rand}}$ | ΔMaxAr |
|---|---|---|---|---|---|---|
| English | 3.79 | 7.07 | −3.28 | 4.97 | 2.82 | +2.15 |
| Hindi | 4.25 | 8.27 | −4.01 | 5.87 | 3.08 | +2.78 |
| German | 3.82 | 7.64 | −3.82 | 5.31 | 2.98 | +2.33 |
| Japanese | 4.33 | 8.68 | −4.35 | 5.51 | 3.08 | +2.43 |
| Tamil | 4.31 | 6.96 | −2.65 | 4.95 | 2.85 | +2.11 |
| Sanskrit | 2.97 | 3.93 | −0.96 | 3.02 | 2.17 | +0.85 |

### 4.2 Depth Distributions (H1)

Figure 1 shows depth distributions for all six languages. In every panel the real (blue) distribution is shifted markedly left of the random (red) distribution. Real trees cluster at low depths (mode 3–5) while random trees spread widely with long right tails reaching depth 20+. Head-final languages (Hindi, Japanese) produce slightly deeper real trees than English, consistent with their left-branching structure, yet all remain far below random baselines.

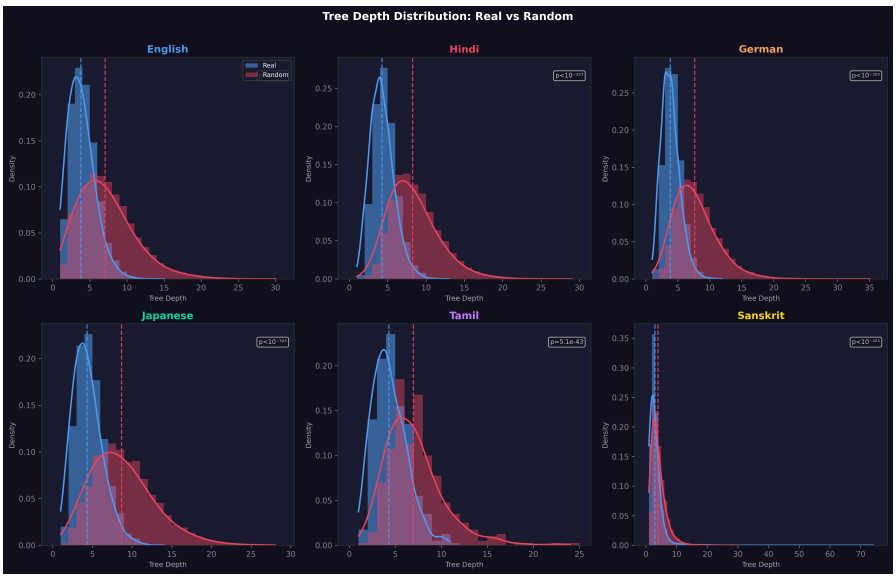


***Figure 1.*** *Depth distributions (real = blue, random = red) across six languages. KDE overlay; dashed lines mark distribution means.*

### 4.3 Max Arity Distributions (H2)

Figure 2 confirms the depth-breadth tradeoff: real trees consistently have higher MaxArity than random trees. The main verb in natural language gathers many direct dependents at one level, widening the tree rather than deepening it.

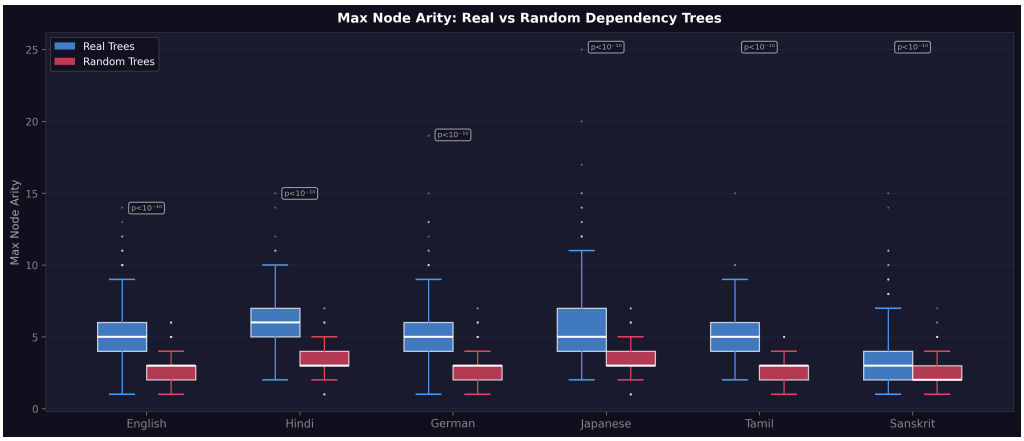


***Figure 2.*** *MaxArity boxplots (real vs. random) per language. Box = IQR; whiskers = 1.5×IQR; dots = outliers.*

### 4.4 Cross-Linguistic Comparison

Figure 3 shows mean depth (left) and mean MaxArity (right) across all languages. The consistent separation between real and random bars — present in every language for both metrics — confirms that both constraints operate universally, not only in well-studied Indo-European languages.

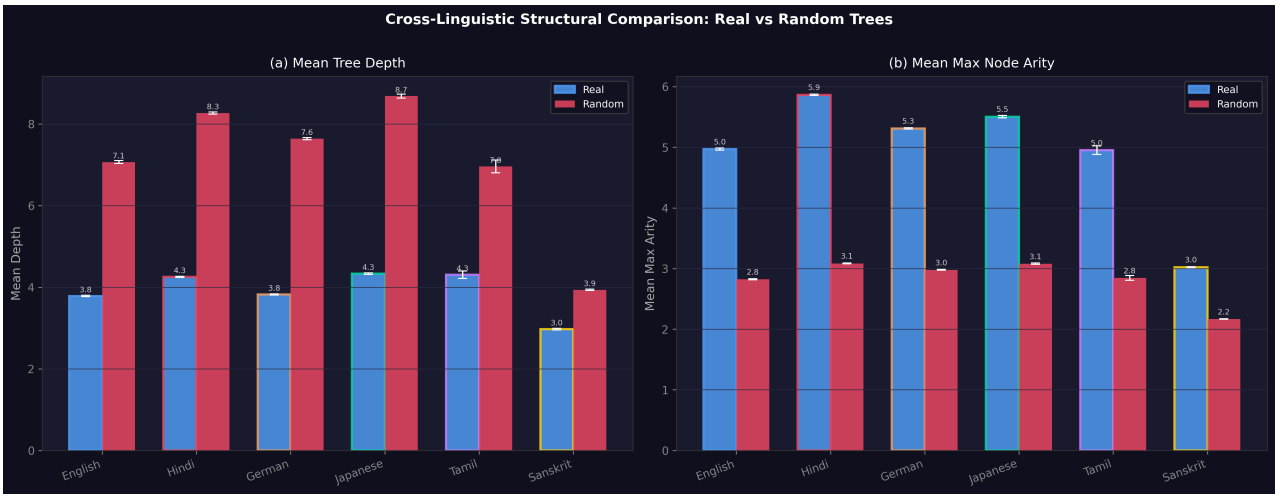


***Figure 3.*** *Cross-linguistic comparison. Blue = real; Red = random. Error bars = ±1 SEM.*

### 4.5 Depth Scales Sublinearly with Sentence Length

Figure 4 plots mean depth against sentence length. In all six languages real trees grow much more slowly in depth as sentences get longer than random trees do — confirming that the depth constraint is not a sentence-length artifact but a structural property maintained even as sentences grow.

### 4.6 Statistical Results

Table 3 and Figure 5 report the full test results. All 12 tests are significant at the Bonferroni-adjusted threshold $\alpha' = 0.00417$. Effect sizes are uniformly large ($|d| > 1.0$) for all languages except Sanskrit (small to medium), indicating practically meaningful differences, not merely statistically detectable ones.

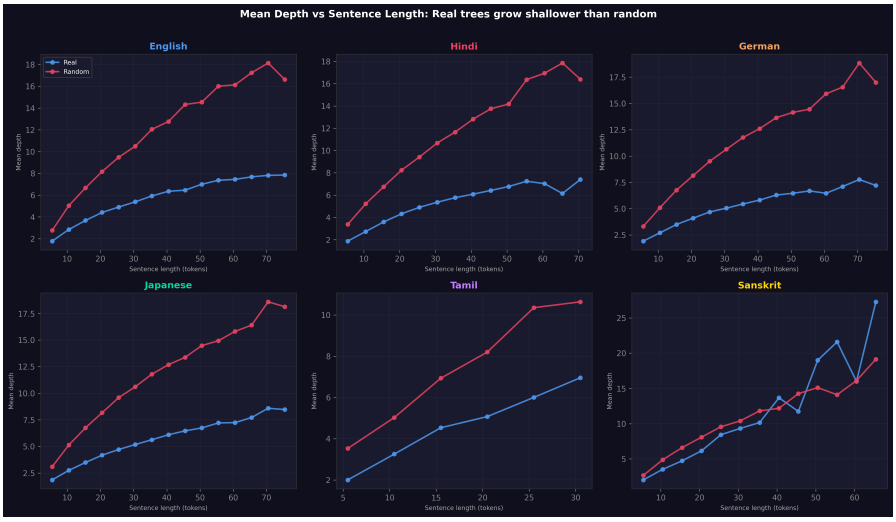


***Figure 4.*** *Mean depth vs. sentence length (binned). Real trees (blue) grow sublinearly; random trees (red) grow much faster.*

***Table 3.*** *Full statistical results. All 12 tests significant at $\alpha' = 0.00417$. Negative $d$ (H1) = real shallower; Positive $d$ (H2) = real wider.*

| Language | Hyp | $\bar{x}_{\text{real}}$ | $\bar{x}_{\text{rand}}$ | $\Delta$ | $p$-value | Cohen $d$ | Sig. |
|---|---|---|---|---|---|---|---|
| English | H1 | 3.79 | 7.07 | $-3.28$ | $< 10^{-323}$ | $-1.12$ | ✓ |
| | H2 | 4.97 | 2.82 | $+2.15$ | $< 10^{-323}$ | $+1.55$ | ✓ |
| Hindi | H1 | 4.25 | 8.27 | $-4.01$ | $< 10^{-323}$ | $-1.61$ | ✓ |
| | H2 | 5.87 | 3.08 | $+2.78$ | $< 10^{-323}$ | $+2.49$ | ✓ |
| German | H1 | 3.82 | 7.64 | $-3.82$ | $< 10^{-323}$ | $-1.52$ | ✓ |
| | H2 | 5.31 | 2.98 | $+2.33$ | $< 10^{-323}$ | $+2.01$ | ✓ |
| Japanese | H1 | 4.33 | 8.68 | $-4.35$ | $< 10^{-323}$ | $-1.41$ | ✓ |
| | H2 | 5.51 | 3.08 | $+2.43$ | $< 10^{-323}$ | $+1.74$ | ✓ |
| Tamil | H1 | 4.31 | 6.96 | $-2.65$ | $5.1 \times 10^{-43}$ | $-1.03$ | ✓ |
| | H2 | 4.95 | 2.85 | $+2.11$ | $4.8 \times 10^{-96}$ | $+1.86$ | ✓ |
| Sanskrit | H1 | 2.97 | 3.93 | $-0.96$ | $< 10^{-323}$ | $-0.38$ | ✓ |
| | H2 | 3.02 | 2.17 | $+0.85$ | $< 10^{-323}$ | $+0.75$ | ✓ |

## 5. Theoretical Implications

### 5.1 Universal Depth Constraints and Dependency Length Minimization

H1 is confirmed in all six languages, including typologically distant ones (Dravidian Tamil, Japonic Japanese, and the ancient Indo-Aryan Sanskrit). This cross-linguistic universality is consistent with **Dependency Length Minimization** (DLM; [2, 11, 3]): speakers prefer to keep syntactically related words close, which naturally limits recursive embedding and keeps trees shallow. The finding holds even in Sanskrit, a free word-order language — though with a smaller effect ($|d| = 0.38$, 'small') — suggesting DLM operates universally but is modulated by morphological richness. Rich morphological case marking in Sanskrit allows semantic roles to be recovered from inflection rather than word order, giving speakers more flexibility to "escape" local proximity pressure.

The sublinear depth-vs-length relationship (Figure 4) further supports this: even as sentences become longer, speakers do not simply add embedding levels. Instead they broaden trees, adding more material at existing depth levels. This is exactly what working memory constraints predict [5]: maintaining many open syntactic expectations across deep embedding is costly, so humans prefer flat structures.

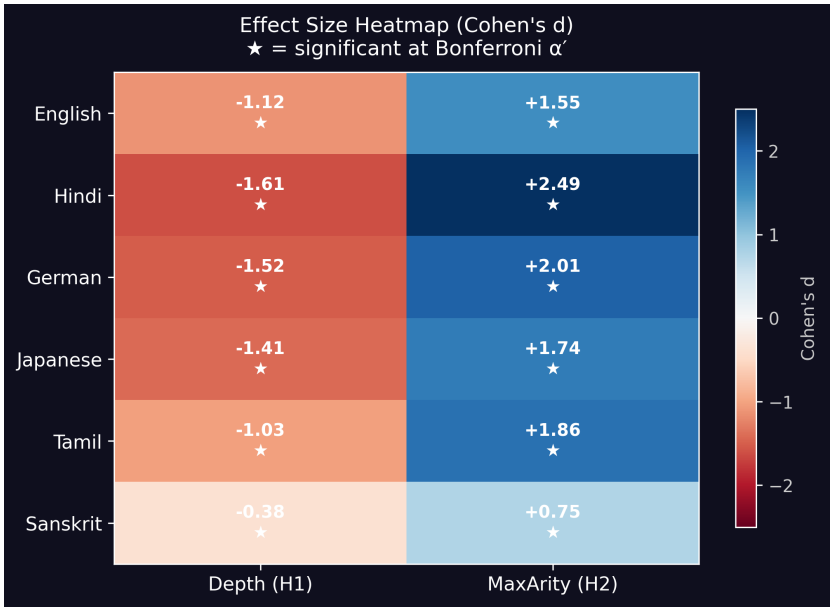


***Figure 5.*** *Cohen's d heatmap across all language × metric combinations. Blue (H1, negative d) = real shallower; Red (H2, positive d) = real wider. ⋆ = significant at Bonferroni-adjusted $\alpha'$.*

### 5.2 The Depth-Breadth Tradeoff and Cognitive Economy

H2 reveals the structural consequence of the depth constraint. Because real trees are shallow, their heads — especially the root verb — must gather many dependents directly, producing higher MaxArity than random trees. This is confirmed across all six languages ($d > 0.75$ everywhere). The finding aligns with evidence on centre-embedding difficulty [8]: processing a deep tree requires maintaining a long stack of open predictions in working memory, which humans avoid. A wide, flat tree closes dependencies quickly at each level, at the cost of higher local arity — a tradeoff that appears to be the cognitively preferred solution.

For theories of **language evolution**, these results suggest that the depth constraint may be a product of iterated learning under processing pressure [4]: languages that systematically produce deep trees are harder to process and transmit, leading to gradual flattening over generations.

### 5.3 Implications for Natural Language Processing

The large effect sizes ($|d| > 1.0$ in 10/12 tests) indicate that depth and MaxArity are highly informative structural features. Dependency parsers that incorporate priors consistent with these distributions should improve parse selection in ambiguous sentences. The cross-linguistic consistency of the constraints suggests that multilingual parsers could share structural priors across languages, improving zero-shot transfer.

### 5.4 Limitations and Future Work

Tamil has only 400 sentences; results should be interpreted cautiously. The Prüfer baseline assumes uniformly random trees; more constrained baselines (e.g., random projective trees) could reveal finer-grained constraints. Future work should extend the analysis to 20+ languages and examine additional structural properties such as projectivity, branching direction, and subtree balance.

## References


[1] Fromkin, V., Rodman, R., & Hyams, N. (2007). *An Introduction to Language*. Thomson Wadsworth.

[2] Gibson, E. (1998). Linguistic complexity: Locality of syntactic dependencies. *Cognition*, 68(1), 1–76.

[3] Gildea, D., & Jaeger, T. F. (2015). Human languages order information efficiently. *arXiv*:1510.02823.

[4] Kirby, S., Tamariz, M., Cornish, H., & Smith, K. (2015). Compression and communication in the cultural evolution of linguistic structure. *Cognition*, 141, 87–102.

[5] Lewis, R. L., Vasishth, S., & Van Dyke, J. A. (2006). Computational principles of working memory in sentence comprehension. *Trends in Cognitive Sciences*, 10(10), 447–454.

[6] de Marneffe, M.-C., & Manning, C. D. (2008). Stanford typed dependencies manual. *Technical Report*, Stanford University.

[7] McDonald, R., Pereira, F., Ribarov, K., & Hajič, J. (2005). Non-projective dependency parsing using spanning tree algorithms. In *Proc. HLT-EMNLP* (pp. 523–530).

[8] Miller, G. A., & Chomsky, N. (1963). Finitary models of language users. In Luce, Bush & Galanter (eds.), *Handbook of Mathematical Psychology*, Vol. 2.

[9] Nivre, J., et al. (2016). Universal Dependencies v1: A multilingual treebank collection. In *Proc. LREC*.

[10] Prüfer, H. (1918). Neuer Beweis eines Satzes über Permutationen. *Archiv der Mathematik und Physik*, 27, 742–744.

[11] Temperley, D. (2007). Minimization of dependency length in written English. *Cognition*, 105(2), 300–333.

[12] Universal Dependencies Project (2024). https://universaldependencies.org

[13] GitHub Repository

# A. Implementation Details

## A.1 Language Selection and Data Constraints

At the start of this project, I wanted to include Kusunda, a critically endangered language spoken in Nepal. Since Kusunda is a language isolate, it seemed like a very interesting case in testing whether syntactic patterns observed in other languages are truly universal. However, we soon realized that it is an extremely low-resource language, and there are no available datasets in the Universal Dependencies format. Because of this, I was unable to include it in our analysis.

We then looked at other languages such as Bhojpuri, Magahi, and Prakrit. These languages are widely spoken and linguistically important, so they would have been good additions to the study. However, I started facing a similar issue here as well—none of them currently have standardized dependency treebanks in the Universal Dependencies framework. Without consistent and sufficiently large annotated data, it was not possible to include them in a systematic comparison.

We also considered creating a small dependency dataset ourselves. However, dependency annotation takes a significant amount of time and requires careful adherence to guidelines to ensure consistency. Given the limited scope of this project, a small manually created dataset would not have been sufficient for statistical analysis compared to well established languages

For these reasons, I turned my focus only on languages that already have well-established Universal Dependencies treebanks. The complete implementation is provided in `dag_real_ud.ipynb`. All code is Python 3.12 using standard libraries (`networkx`, `numpy`, `scipy`, `matplotlib`, `seaborn`). No external parser library is needed. To reproduce all results, place the six `.conllu` files in the same directory as the notebook and run all cells top-to-bottom.

## A.2 CoNLL-U Parser

***Listing 1.*** *Custom CoNLL-U parser (Cell 2 of notebook).*

```python
def parse_conllu(filepath, min_len=3, max_len=80):
    """
    Parse a CoNLL-U file into a list of networkx DiGraph trees.
    Nodes = word indices (1-based); edges = head -> dependent.
    """
    trees = []
    with open(filepath, encoding='utf-8') as f:
        raw = f.read()

    for block in raw.strip().split('\n\n'):
        G = nx.DiGraph()
        valid = True
        for line in block.strip().split('\n'):
            if line.startswith('#'):       # comment line
                continue
            parts = line.split('\t')
            if len(parts) < 8:
                continue
            tok_id = parts[0]
            # skip multiword tokens (e.g. "1-2") and empty nodes ("1.1")
            if '-' in tok_id or '.' in tok_id:
                continue
            try:
                tid  = int(tok_id)
                head = int(parts[6])       # HEAD column
            except ValueError:
                valid = False; break
            G.add_node(tid)
            if head != 0:
                G.add_edge(head, tid)      # directed: head -> dependent
        n = len(G.nodes())
        if valid and min_len <= n <= max_len \
                 and nx.is_directed_acyclic_graph(G):
            trees.append(G)
    return trees
```

## A.3 Prüfer Sequence Random Tree Generator

***Listing 2.*** *Prüfer decoder and random tree generator (Cell 3 of notebook).*

```python
def prufer_decode(sequence):
    """Decode Prufer sequence -> list of (u,v) undirected edges."""
    n      = len(sequence) + 2
    degree = [1] * (n + 1)
    for node in sequence:
        degree[node] += 1
    edges = []
    for node in sequence:
        for leaf in range(1, n + 1):
            if degree[leaf] == 1:
                edges.append((node, leaf))
                degree[node] -= 1
                degree[leaf] -= 1
                break
    remaining = [v for v in range(1, n + 1) if degree[v] == 1]
    edges.append((remaining[0], remaining[1]))
    return edges

def random_tree_prufer(n):
    """Uniformly random rooted directed tree on n nodes."""
    if n <= 2:
        G = nx.DiGraph()
        if n == 2: G.add_edge(1, 2)
        else:      G.add_node(1)
        return G
    seq     = [random.randint(1, n) for _ in range(n - 2)]
    edges   = prufer_decode(seq)
    G_undir = nx.Graph()
    G_undir.add_edges_from(edges)
    root    = random.randint(1, n)
    return nx.bfs_tree(G_undir, root)   # oriented: parent -> child
```

## A.4 Metric Computation

***Listing 3.*** *Depth and MaxArity computation (Cell 4 of notebook).*

```python
def compute_metrics(G):
    """
    Returns dict with:
      depth     : max distance root -> any node  (H1 metric)
      max_arity : max out-degree of any node     (H2 metric)
    """
    roots = [n for n, d in G.in_degree() if d == 0]
    if len(roots) != 1: return None          # malformed tree
    root      = roots[0]
    dist      = nx.single_source_shortest_path_length(G, root)
    depth     = max(dist.values())
    max_arity = max(d for _, d in G.out_degree())
    return {'n': len(G.nodes()), 'depth': depth, 'max_arity': max_arity}
```

## A.5 Statistical Testing

***Listing 4.*** *Mann-Whitney U test with Bonferroni correction (Cell 6 of notebook).*

```python
from scipy import stats
import math, numpy as np

ALPHA_ADJ = 0.05 / 12   # Bonferroni: 6 languages x 2 metrics

# H1: one-sided 'less' (real depth < random depth)
u1, p1 = stats.mannwhitneyu(
    real_depths, rand_depths, alternative='less')

# H2: one-sided 'greater' (real MaxArity > random MaxArity)
u2, p2 = stats.mannwhitneyu(
    real_maxar, rand_maxar, alternative='greater')

def cohens_d(a, b):
    s_pool = math.sqrt((np.var(a, ddof=1) + np.var(b, ddof=1)) / 2)
    return (np.mean(a) - np.mean(b)) / s_pool

d1 = cohens_d(real_depths, rand_depths)     # expect negative (H1)
d2 = cohens_d(real_maxar,  rand_maxar)      # expect positive (H2)
```